Sistemas Operativos I

Programas Concurrentes

En esta clase introduciremos varios conceptos:

- Proceso vs Programa
- Vida de una Proceso
- Estructuras
- Programas Concurrentes
- Garantías del SO
- Programación en C: Fork/Exec/Wait/Exit

Procesos vs Programas

Programa

Instrucciones escritas en algún lenguaje de programación.
Es totalmente estático, es texto en un lenguaje.

Proceso

Es el programa en acción.
La entidad **dinámica** generada por la ejecución de un programa en **uno o varios** procesadores.

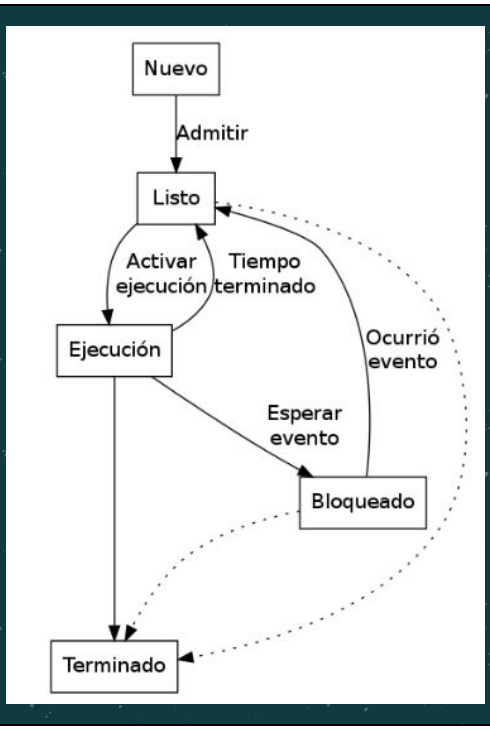
El Sistema Operativo brinda la ilusión de que varios procesos pueden ejecutarse simultáneamente, pero en realidad, es la cantidad de hardware subyacente quien dicta realmente cuántos procesos pueden estar ejecutándose simultáneamente.

Un proceso NO es sólo un programa en ejecución. Un proceso incluye también:

- Los Archivos abiertos
- las señales (signals) pendientes
- Datos internos del kernel
- El estado completo del procesador
- Un espacio de direcciones de memoria
- Uno o más hilos de Ejecución. Cada thread contiene:
    - Un único contador de programa
    - Un Stack
    - Un Conjunto de Registros
- Una sección de datos globales

La vida de un Proceso

- Nuevo
- Listo
- Ejecución
- Bloqueado
- Terminado
- Zombie

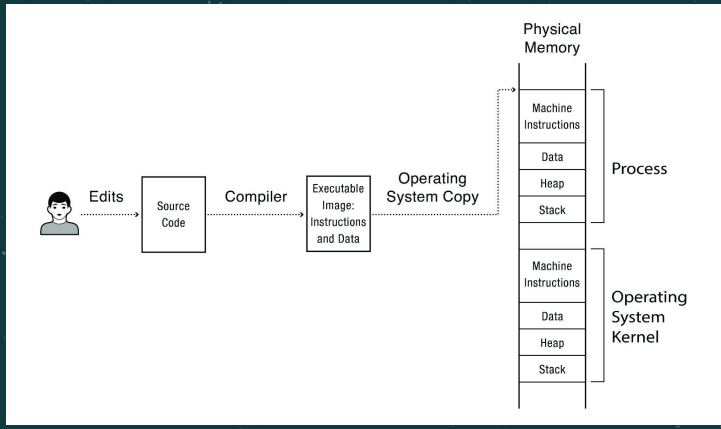


+ Nuevo: Se solicitó al SO la creación de un proceso cuyo recursos y estructuras están siendo creadas
+ Listo: Está listo para iniciar la ejecución pero el SO no le asignó ningún procesador
+ En Ejecución: El proceso está siendo ejecutado en éste momento.
+ Bloqueado: En espera de un evento para poder continuar su ejecución
+ Terminado: El proceso terminó de ejecutarse y sus estructuras están a la espera de ser eliminadas por el SO.
+ Zombie: El programa ha terminado su ejecución pero el SO todavía debe realizar algunas operaciones de limpieza para poder eliminarlo. Notificar al proceso padre, cerrar conexiones de red, liberar memoria, etc. Programas que terminaron (con `exit`) pero todavía se guarda información de ellos.

Áreas de Memoria que Ocupa un Proceso (repaso de Arquitectura)

Los procesos están dividido en 4 segmentos:

- **Text:** Instrucciones del Programa
- **Data:** Variables Globales (extern o static)
- **Heap:** memoria dinámica (malloc)
- **Stack:** memoria local/temporal dentro de un bloque de código



Text: Instrucciones del Programa.
Data: Variables Globales (extern o static en C). La zona de memoria estática es para datos que no cambian de tamaño, permite almacenar variables globales que son persistentes durante la ejecución de un programa.
Heap: permite almacenar variables adquiridas dinámicamente (vía funciones como malloc o calloc) durante la ejecución de un programa.
Stack: permite almacenar argumentos y variables locales durante la ejecución de las funciones en las que están definidas.

Información de un Proceso que lleva el SO

El Kernel para cada proceso tiene un PCB (Process Control Block) que contiene la siguiente información:

- Identificador del Proceso
- Estado del Proceso
- Contador del Programa (siguiente instrucción a ejecutar)
- Registros del CPU que está utilizando
- Información de la memoria que está utilizando
- File Descriptor Table (dispositivos de entrada y salida que el proceso ha abierto)

El sistema operativo lleva información del estado de ejecución de cada proceso:

+ Identificador del Proceso
+ El estado de ejecución, nuevo, listo, en ejecución, etc
+ La siguiente instrucción a ejecutar
+ Que registros del cpu está utilizando
+ información de la memoria que está utilizando
+ File Descriptor Table (Lista de dispositivos de entrada y salida que el proceso ha abierto)

Concurrencia

Programa Concurrente es la descripción de un conjunto de máquinas de estados que cooperan mediante un medio de comunicación.
Dos procesos se ejecutan concurrentemente si comparten tiempo de vida.

La **programación concurrente** hace referencia a técnicas de programación que son utilizadas para expresar la concurrencia entre tareas y solución de los problemas de comunicación y sincronización entre procesos.

Un programa concurrente es la descripción de un algoritmo mediante la cooperación de un conjunto de máquinas de estados que se comunican de alguna manera.
Dos procesos se dicen concurrentes si comparten en (algún momento) un tiempo de vida. **No quiere decir que se ejecuten simultáneamente**, uno puede estar listo mientras otro puede estar en ejecución, etc. Para que se ejecuten en simultáneo es necesario contar con el hardware para hacerlo. La programación paralela se basa en la idea de la ejecución de procesos de forma simultánea, que entra bajo la definición de procesos concurrentes.

Garantías que asumimos del Sistema Operativo

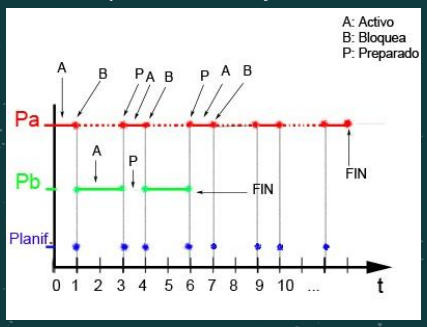
- Scheduler Justo (todo proceso listo eventualmente se ejecutará)
- Procesos Confiables (Los procesos harán lo que el programa les dice qué hacer)
- **Procesos Asíncronos** (No se puede asumir ningún orden de ejecución entre procesos)

En el caso que tengamos menos procesadores (unidades de procesamiento) físicos que procesos tendremos que seleccionar qué procesos se ejecutarán en cada momento. Eso lo hace el Sistema Operativo, y está totalmente oculto a los procesos (y nosotros).

+ El scheduler (o administrador de procesos) del Sistema Operativo es el encargado de seleccionar qué procesos, dentro del conjunto de procesos listos, comenzarán la ejecución. Nosotros asumimos que el scheduler es justo en el sentido que todo proceso listo eventualmente se ejecutará.
+ Los procesos harán lo que el programa les dice que hacer, y podemos confiar que su ejecución es fiel.
+ Además, no asumimos ninguna restriccion de tiempo sobre la ejecución de las operaciones de cada uno de los procesos. En particular, no podemos asumir ningún orden de ejecución entre procesos concurrentes.

Scheduler (Planificador de Procesos)

- El Scheduler, es un módulo dentro del Kernel del SO, se encarga de seleccionar a qué proceso se asigna el recurso procesador y durante cuánto tiempo.



El planificador de procesos tiene como funciones principales:

- Llevar el control del estado de cada proceso.
- Decidir qué proceso usará el procesador y durante cuánto tiempo, para ello emplea un cierto criterio en base al cual tomará las decisiones. Si implementa multiprogramación, deberá emplear un criterio que evite que un proceso monopolice el procesador.
- Asignar el procesador al proceso, restableciendo el contexto de la ejecución del proceso por donde fuere. El contexto de la ejecución se encuentra almacenado en el PCB del proceso asignado.
- Retirar el procesador al proceso, salvando el contexto de la ejecución del proceso, para poder restablecerlo posteriormente una vez que vuelva a ser asignado. El contexto de la ejecución se almacenará en el PCB del proceso retirado.

El planificador de procesos es software, por tanto, requiere emplear el recurso procesador para ejecutar su código.

Para la siguiente configuración de procesos:

- Proceso Pa: Llega en el instante de tiempo 0. Realiza una llamada al sistema bloqueante cada 1 unidad de tiempo, la operación bloqueante se resuelve tras 2 unidades de tiempo. Para finalizar su ejecución requiere 5 unidades de ejecución.
- Proceso Pb: Llega en el instante de tiempo 1. Para finalizar su ejecución requiere 4 unidades de ejecución.
- Suponga que el proceso Pa tiene mayor prioridad que el proceso Pb, por

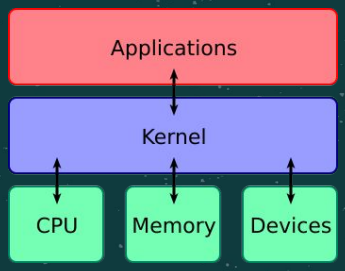
- tanto, el planificador debe de dejar paso al proceso Pa siempre que éste esté en estado preparado.

Cambio de Contexto (Context Switch)

- El cambio de contexto es el proceso por el cual el SO transfiere el control de la CPU de un proceso (o thread) a otro. Esto implica guardar el estado del proceso que se está ejecutando actualmente y cargar el estado del proceso al que se va a dar paso.
- Un cambio de contexto puede darse por
  - Agotamiento del tiempo de ejecución
  - Interrupciones (Ejemplo: enviar una señal a un Proceso para que termine SIGKILL)
  - Preemption (El Scheduler quita el proceso de ejecución porque otro tiene mayor prioridad)

Kernel

- El Kernel (núcleo) facilita a los distintos procesos acceso seguro al hardware (CPU, memoria, E/S) de la computadora. Gestiona dichos recursos a través de servicios de llamada al sistema,



Programación en C!

Llamada a sistema para creación de procesos desde C!

Vamos a pasar a aplicar los conceptos que aprendimos hoy.
Veremos cómo crear procesos mediante llamadas a sistema, en particular dos formas de hacerlo. Veremos formas primitivas de compartir información.

UNISTD.h

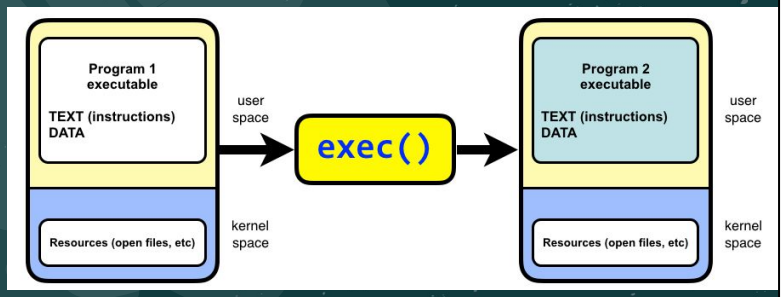
El archivo de cabecera unistd.h nos ofrece un punto de acceso a la *Portable Operating System Interface* (POSIX) del sistema operativo

- `close / open`
- `read / write`
- `exec / fork / getpid`
- `select / pipe`

Comunicación con el Sistema Operativo por medio de la librería Unistd.
En particular, hoy utilizaremos **exec, fork and getpid.**

Exec

Reemplaza la imagen de programa del proceso actual por una nueva. Es decir, termina la ejecución del programa actual, y comienza la de otro **sin cambiar de Identificación de proceso**.



Notar que aquí no hay comunicación alguna, cuando un proceso invoca a una función de la familia de exec su programa es reemplazo, y otro se ejecuta en su lugar.

Cuando se llama a Exec si bien el proceso es reemplazado, los file descriptors se mantienen. El comportamiento por defecto es de Herencia. Por defecto, los archivos permanecen abiertos. El nuevo programa hereda todos los descriptores de archivo que estaban abiertos en el proceso original (stdin, stdout, stderr y cualquier otro archivo o socket). Esto es fundamental para el funcionamiento de los sistemas tipo Unix. Por ejemplo, cuando una shell redirige la salida de un comando a un archivo (ls > lista.txt), la shell abre el archivo y luego llama a exec. Como el descriptor se mantiene abierto, el nuevo programa (ls) escribe en el archivo sin siquiera saber que fue redirigido.

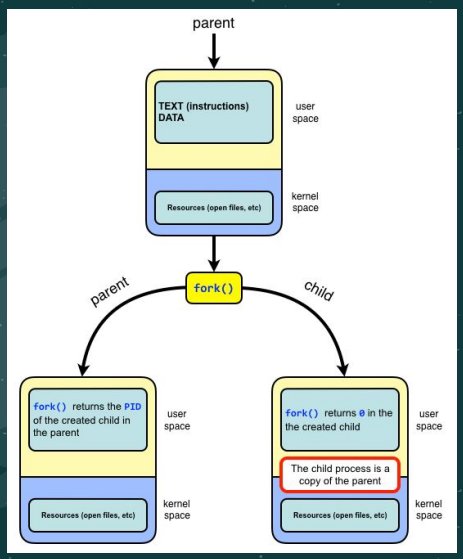
Ver cómo funciona getpid()

observar que se pasa dos veces el path al binario (binaryPath) en execl(binaryPath, binaryPath, arg1, arg2, NULL) porque el primero es la ruta del ejecutable y el otro hace referencia al primer argumento que puede tomar un programa en C. El primer argumento que puede tomar un programa en C, está dado por main(int argc, char** argv) y es argv[0] que hace referencia al nombre del programa.

Fork

Se crea una **copia** del proceso que invoca a **fork** pero en un nuevo espacio de memoria.
El proceso que invoca a fork se lo denomina **parent** mientras que el proceso creado por la invocación se denomina **child**.
Tenemos 3 resultados posibles a la llamada fork

- -1 indicando un error
- 0 indicando que es el proceso child
- el PID (>0) del proceso child



Es decir, el proceso Parent sabe quien es porque el resultado de fork() es el PID del proceso child (y siempre es mayor a 0), mientras que el proceso Child no sabe quién es el Parent porque el resultado de invocar a fork() es 0!
Notar que la relación es asimétrica, por eso se llama Parent/Child, porque el proceso Parent sabe quien és el proceso Child mientras que es el proceso Child no siempre sabe quién es el proceso Parent. Y se utiliza la nomenclatura Parent/Child para establecer la cronología, Parent viene desde antes que Child, y es quien *lo crea.*

Por fin tenemos procesos compartiendo vida! Es decir, procesos concurrentes!!

Fork: los procesos NO comparten memoria.
Fork: los procesos si comparten la tabla de File Descriptors, esto es, ven los mismos archivos, pipes, E/S standard.
Fork: Se hace la copia de la memoria utilizando la técnica Copy-on-write. La memoria solo se copia bajo demanda. Esto es, la memoria se copia cuando el proceso Child quiere modificarla.

EJEMPLO!

Proceso que crea otro con fork, y mostramos que los PIDs varían!

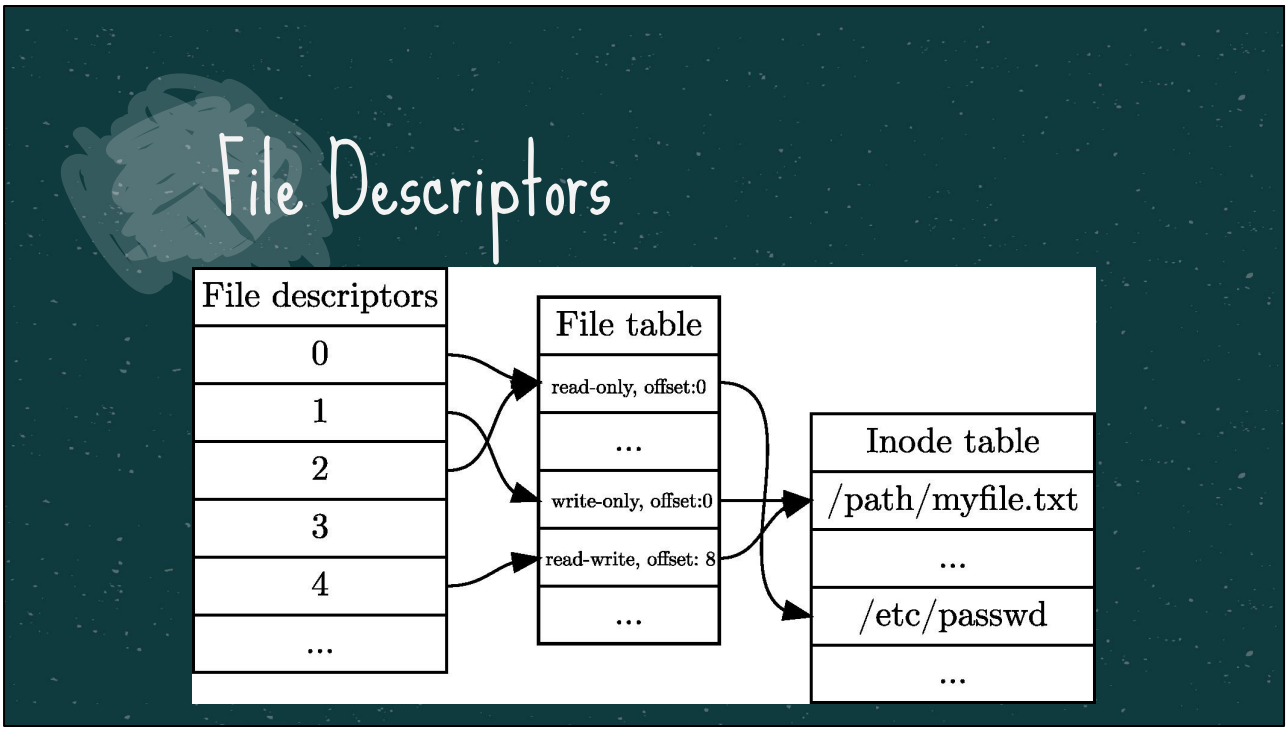
Notas sobre fork()

- El Child tendrá su propio ID de proceso único.
- Los procesos Parent/Child **NO comparten memoria**.
- Los procesos tienen una copia de la **tabla de File Descriptors**, esto es, ven los mismos archivos, pipes, E/S standard. Más adelante vamos a ver que pueden utilizar los Files Descriptors para comunicarse entre sí (***Inter-Process Communication***).
- La copia de memoria se hace utilizando la técnica ***Copy-on-write***. La memoria solo se copia bajo demanda. Esto es, la memoria se copia cuando el proceso Child quiere modificarla.

EJEMPLO!

¿Cómo se comporta el siguiente programa?

```
int main()
{
  pid_t pid1 = fork();
  pid_t pid2 = fork();
  printf("pid1: %d pid2: %d!\n", pid1, pid2);
  return 0;
}
```



- Un **File Descriptor** es un unsigned integer utilizado por un proceso para identificar un archivo abierto.
- Para cada proceso en nuestro sistema operativo, hay un Process Control Block (PCB). PCB realiza un seguimiento del contexto del proceso. Entonces, uno de los campos dentro de esto es un array llamado File Descriptor Table.
- **File Descriptor Table:** es la colección de índices de enteros File Descriptors en los que los elementos son punteros a elementos de la **File Table**. Para cada proceso, se proporciona una File Descriptor Table única en el sistema operativo.
- La **File Table** es una **estructura de datos residente en el kernel**, que contiene detalles de todos los archivos abiertos. Un elemento en la **File Table** es una estructura en memoria para un archivo/recurso abierto, que se crea cuando se procesa una solicitud para abrir un archivo/recurso y estas entradas mantienen la posición del archivo/recurso. Los archivos/recursos podrían ser:

- File
- E/S de terminal
- Pipes
- Socket (para el canal de comunicación entre computadoras)
- Cualquier dispositivo (CD-ROM, USB, etc)

File Descriptors

- Cada proceso en ejecución comienza con tres archivos ya abiertos:

Nombre del File Descriptor	**Nombre corto**	**Número del File Descriptor**	**Descripción**
Entrada estándar	stdin	0	Entrada desde el teclado
Salida estándar	stdout	1	Salida a la consola
Salida de error estándar	stderr	2	Salida de error a la consola

creat/open

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
int creat(const char *pathname, mode_t mode);
```

- El valor de retorno de open() es un File Descriptor, que se puede utilizar por llamadas al sistema como read(2), write(2), lseek(2), fcntl(2), etc, para hacer referencia al archivo abierto. El File Descriptor devuelto es el número más bajo que no esté abierto actualmente para el proceso.

read/write/close

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
int close(int fd);
```

- **read()** intenta leer hasta contar bytes desde el descriptor de archivo fd en el búfer a partir de buf.
- En caso de éxito, se devuelve el número de bytes leídos (0 indica el final del archivo) y la posición del archivo avanza en este número. No es un error si este número es menor que el número de bytes solicitados; esto puede suceder, por ejemplo, porque hay menos bytes disponibles en este momento (tal vez porque estábamos cerca del final del archivo, o porque estamos leyendo de un pipe, o de un terminal), o porque read() fue interrumpido por una señal.

Ejercicio!

Abrir y leer de un archivo 1024 bytes.

Volvamos a trabajar con Procesos concurrentes

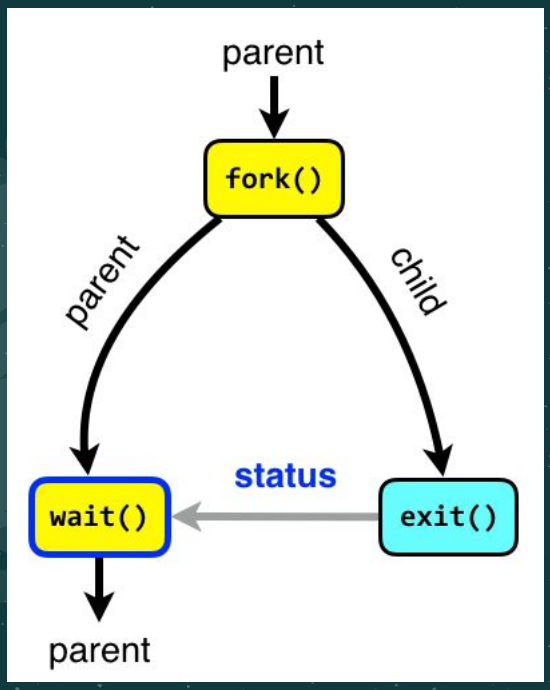
Ejercicio!

Escribir un programa en C que toma como entrada la dirección a un binario, un tiempo en segundos, y ejecuta el comando cada los segundo especificados.

Función útil **sleep** debería ser útil

Wait

Cuando un proceso parent crea un proceso *child* éste puede esperar el resultado de la ejecución del proceso child.



wait() es bloqueante. Si el Child ya terminó y el Parent

Si un Child termina, pero que no ha sido esperado (wait), se convierte en un "zombie".
El kernel mantiene un conjunto mínimo de información sobre el proceso zombie (PID, estado de terminación, información de uso de recursos).
Para permitir que el Parent realice una espera para obtener información sobre el Child. Siempre que un zombi no se elimine del sistema a través de una espera, consumirá un slot en la tabla de procesos del kernel, y si esta tabla se llena, no será posible crear más procesos. Si un proceso padre termina, sus hijos "zombie" (si los hay) son adoptados por init(1), y este realiza automáticamente una espera para eliminar los zombis.

EJEMPLO!

Proceso que crea otro con fork, el proceso parent espera a que finalice el proceso child!

Qué pasa si el Parent termina antes que el Child?

- Si el Parent termina antes que los Children, los Children se convierten en Children del proceso **Init** (que tiene el ID de proceso **1**, y el kernel lo inicia como el primer proceso de usuario).
- El proceso **init** comprueba periódicamente si hay nuevos Children y hace un wait (liberando así los recursos que se asignan a cada Child).
- Si un Child termina, pero que no ha sido esperado (wait), se convierte en un **zombie**. El kernel mantiene un conjunto mínimo de información sobre el proceso zombie (PID, estado de terminación, información de uso de recursos) de manera de permitir después al Parent poder hacer un wait para obtener información sobre el Child. Siempre que un zombie no se elimine del sistema a través de un wait consumirá un slot en la tabla de procesos del kernel, y si esta tabla se llena, no será posible crear más procesos. Si un proceso Parent termina, sus Children zombies (si los hay) son adoptados por init(1), y este realiza automáticamente un wait para eliminar los zombies.

Ejercicio!

Hacer una Shell, la misma debe poder leer comandos (con o sin argumentos) ingresados por teclado y ejecutarlo. Una vez finalizado el programa que se ejecutó, debe volver al prompt y permitir el ingreso de otro programa. Se debe también proporcionar el comportamiento de poder salir de la shell.

Sugerencia: utilizar **fgets()** para la lectura de la entrada estándar y **strtok()** para parsear el ejecutable y sus argumentos.